



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Cloud-Based Smart Warehouse Automation

A Simulation-Based Intelligent Warehouse Automation and Digital-Twin Platform

A CAPSTONE PROJECT REPORT

*A Capstone Project Report submitted in partial fulfilment of the requirements
for the Course of*

CSA1501 — Cloud Computing

to the award of the degree of

BACHELOR OF ENGINEERING / BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING

Submitted by

H. Harshavardhan (192511171)

Under the Supervision of

Dr. M. D. Boomija

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences
Chennai-602105

SEPTEMBER 2026

SIMATS ENGINEERING
Saveetha Institute of Medical and Technical Sciences
Chennai-602105

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled “Warehouse OS: Cloud-Based Smart Warehouse Automation” has been carried out by H. Harshavardhan (192511171), Ch. Abhilash Reddy (192525462), Lingamguntla Surya Praneeth (192311233) and Yannamani Sarvarayudu (192324181) under the supervision of Dr. M. D. Boomija and is submitted in partial fulfilment of the requirements for the current semester of the B.Tech / B.E. Computer Science and Engineering programme (Course CSA1501 – Cloud Computing) at Saveetha Institute of Medical and Technical Sciences, Chennai.

COURSE COORDINATOR

DR. A.Gnana soundari

Designation:PROFESSOR

Department of Computer Science and
Engineering/deep learning

SIMATS Engineering, SIMATS, Chennai – 602105.

COURSE FACULTY

Dr. M. D. Boomija,

Designation:PROFESSOR

Department of Computer Science and
Engineering/deep learning

SIMATS Engineering, SIMATS, Chennai – 602105.

DECLARATION

We, H. Harshavardhan, Ch. Abhilash Reddy, Lingamguntla Surya Praneeth and Yannamani Sarvarayudu of the Department of Computer Science and Engineering, SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled “Warehouse OS: Cloud-Based Smart Warehouse Automation” is the result of our own bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with the principles of engineering ethics and academic integrity. All sources, references, data, and other materials used in the project have been appropriately acknowledged. We further declare that the data, results, analysis, and findings presented in this report have not been fabricated, falsified, or manipulated.

Any external tools, software, datasets, computational resources, or AI-assisted tools used during the project — including FastAPI, PostgreSQL, scikit-learn, Google OR-Tools, SimPy, Three.js and the Render cloud platform — have been appropriately disclosed in this report. We confirm that all applicable ethical, academic, institutional, and professional requirements have been duly followed throughout the planning, execution, analysis, and documentation of the project.

Place: Chennai

Date:

Signature of the Students with Names:

1. H. Harshavardhan (192511171)

INDIVIDUAL CONTRIBUTION STATEMENT

(Mandatory for all team projects)

Student	Specific Responsibilities	Design & Development	Testing & Analysis	Report Contribution	Approx . %
H. Harshavardhan (192511171)	Module 1: Core Warehouse Operations & Inventory Management — orders, stock, transfers	FastAPI order/inventory routers, SQLAlchemy models, row-level locking	Order-flow & inventory-accuracy test suite	Ch. 1, Ch. 4 (4.2–4.3 – Module 1)	25%
Ch. Abhilash Reddy (192525462)	Module 2: Security, Access Control & Audit — RBAC, OTP, audit ledger	JWT/OAuth2 auth, bcrypt hashing, hash-chained audit ledger	Security/IDOR test cases, penetration checks	Ch. 3 (3.7), Ch. 4 (4.2 – Module 2)	25%
Lingamguntla Surya Praneeth (192311233)	Module 3: Predictive Intelligence — forecasting & anomaly detection	Polyfit forecasting engine, Isolation Forest model, ABC classification	Forecast accuracy (WAPE/MAE/RMSE) validation	Ch. 2, Ch. 4 (4.2 – Module 3)	25%
Yannamani Sarvarayudu (192324181)	Module 4: Digital Twin, Simulation & AI Assistant	A* pathfinding, Three.js Digital Twin, OR-Tools/SimPy scenario lab, AI assistant	Digital Twin SSE tests, scenario-lab benchmarking	Ch. 4 (4.2 – Module 4), Ch. 5	25%

Total: 100% (equal four-way contribution across the four functional modules; final percentages to be confirmed and signed off by the team and course faculty.)

ABSTRACT

Traditional and semi-automated warehouses frequently suffer from three compounding engineering problems: uncoordinated material-handling robots that collide or deadlock in shared aisles, inventory managers who lack early warning of demand shifts and thus over- or under-stock, and undetected shrinkage (theft, damage, or data-entry error) that erodes margins silently. Warehouse OS addresses this as a single, integrated engineering problem — the absence of one auditable, cloud-deployable platform that unifies core warehouse transactions, autonomous fleet coordination, explainable predictive analytics and tamper-evident security under one data model.

The project's significance lies in demonstrating that a small-scale platform, built on production-grade open-source components, can deliver enterprise-class capability — real-time inventory control, ML-driven forecasting, robot path-planning and a live digital twin — without proprietary WMS licensing costs, while remaining fully auditable for engineering, ethical and security review.

The proposed solution is a decoupled, event-driven client–server system: a FastAPI (Python 3.12, asynchronous) backend with a PostgreSQL/SQLAlchemy persistence layer serves a vanilla-JavaScript single-page frontend rendering a Three.js WebGL digital twin, and communicates over REST and Server-Sent Events. Four functional modules were engineered: (1) Core Warehouse Operations and Inventory Management with row-level locking to prevent race conditions; (2) Security, Access Control and Audit, enforcing JWT/OAuth2 authentication, role-based access control, step-up OTP verification and a SHA-256 hash-chained audit ledger; (3) Predictive Intelligence, combining walk-forward-validated polynomial demand forecasting (scored by WAPE, MAE, RMSE and sMAPE) with Isolation-Forest anomaly detection across four inventory data sources for ABC (Pareto 80/20) classification; and (4) a Digital Twin, Simulation and AI Assistant layer using A* search with a Manhattan-distance heuristic and cell-reservation collision avoidance for AGV routing, Google OR-Tools (CP-SAT) and SimPy for scenario benchmarking, and a natural-language AI Operations Assistant grounded in live RBAC-scoped data.

The methodology followed an iterative build-test-deploy cycle: each module was developed against a documented API contract, validated with an automated pytest suite, profiled for performance, and then verified against a live production deployment on the Render cloud platform. Major results include a reduction of the analytics-dashboard response time from over 30 seconds to under 200 milliseconds after replacing on-demand ML inference with cached background computation; resolution of five major defect classes spanning warehouse CRUD, multi-source ABC analysis, digital-twin streaming authentication and notification integrity; and a local automated test suite of 72 collected cases with 71 passing and 1 skipped (100% pass rate of executed tests), independently re-verified against the live production endpoints for authentication, warehouse management, analytics, anomaly detection, the AI assistant, notifications and the digital-twin API.

The project concludes that a cloud-native, modular architecture combining classical operations-research algorithms (A*, CP-SAT), explainable machine learning (Isolation Forest, validated regression) and cryptographic auditability can be engineered, tested and deployed by a small student team within academic-project constraints, yielding a reproducible, extensible platform suitable as a foundation for further IoT and deep-learning integration.

Keywords: Cloud Warehouse Management System; AGV Fleet Orchestration (A* Pathfinding); Demand Forecasting & Anomaly Detection (Isolation Forest); Digital Twin Simulation; RBAC & Hash-Chained Audit Ledger; FastAPI / PostgreSQL Architecture.

TABLE OF CONTENTS

Sl. No	Title	Page No.
i	BONAFIDE CERTIFICATE	ii
ii	DECLARATION	iii
iii	INDIVIDUAL CONTRIBUTION STATEMENT	iv
iv	ABSTRACT	v
v	LIST OF FIGURES	vii
vi	LIST OF TABLES	viii
vii	LIST OF ABBREVIATIONS / SYMBOLS	ix
1	CHAPTER 1: Introduction and Engineering Problem	1
2	CHAPTER 2: Literature Review and Existing Solutions	4
3	CHAPTER 3: Engineering Design & Trade-offs, Sustainability, Ethics, Safety, Risk & Security	6
4	CHAPTER 4: Implementation, Testing & Results, Project Management	11
5	CHAPTER 5: Conclusion, Future Work and Reflection	23
	References	25
	Appendices	26
	Capstone Project Outcome Mapping Sheet	27

LIST OF FIGURES

Figure No.	Figure Name	Page No.
3.1	System Architecture of Warehouse OS	17
4.1	AI Operations Assistant — Natural-Language Diagnostics Console	24
4.2	Operations Dashboard — Live Warehouse Map & KPIs	25
4.3	Operations Dashboard — Stock-Movement Trend & Alerts	25
4.4	Task Management — Automated Task Feed & Assignment	26
4.5	System Health & Observability Console	26
4.6	Hash-Chained Audit Log — Ledger Integrity View	27
4.7	Users & Roles — RBAC Permission Matrix	27
4.8	Scenario Lab — What-If Experiment Configuration	28
4.9	Scenario Lab — Simulation Execution & Comparison Metrics	28
4.10	Pathfinding & Route Optimization — Interactive Warehouse Grid	29
4.11	3D Digital Twin — Live Robot Positions and Task Heatmap	29
4.12	AI Decision Center — Recommendation with Human Approval Sign-off	30

LIST OF TABLES

Table No.	Table Name
2.1	Comparative Analysis of Existing Warehouse Automation Approaches
3.1	Stakeholder / User Requirements
3.2	Functional & Non-Functional Requirements
3.3	Constraints & Applicable Standards
3.4	Design Specifications
3.5	Alternative Solutions — Evaluation Matrix
3.6	Trade-off Analysis
3.7	Sustainability, Ethics, Safety & Security Evaluation
3.8	Risk Register
4.1	Core Technology Stack
4.2	Tools / Software / Equipment Used
4.3	Major Bug Fixes & Performance Optimizations
4.4	Local Automated Test Suite Summary
4.5	Live Production Verification Benchmark
4.6	Test Plan & Verification
4.7	Comparison with Existing Solutions & Achievement of Objectives
4.8	Project Roles & Contribution Table
4.9	Indicative Project Cost Table

LIST OF ABBREVIATIONS / SYMBOLS

Symbol	Description	Unit
AGV	Automated Guided Vehicle	—
API	Application Programming Interface	—
ABC	Always Better Control (Pareto-based inventory classification)	—
CP-SAT	Constraint Programming – Satisfiability (OR-Tools solver)	—
CRUD	Create, Read, Update, Delete	—
IDOR	Insecure Direct Object Reference	—
JWT	JSON Web Token	—
MAE	Mean Absolute Error	—
OTP	One-Time Password	—
RBAC	Role-Based Access Control	—
RMSE	Root Mean Square Error	—
SKU	Stock Keeping Unit	—
sMAPE	Symmetric Mean Absolute Percentage Error	—
SPA	Single Page Application	—
SSE	Server-Sent Events	—
WAPE	Weighted Absolute Percentage Error	—
WMS	Warehouse Management System	—

CHAPTER 1: INTRODUCTION AND ENGINEERING PROBLEM

1.1 Background

Warehousing sits at the operational core of modern retail and logistics supply chains, and its efficiency directly determines order-fulfilment speed, inventory carrying cost, and customer satisfaction. Classical Warehouse Management Systems (WMS) were built as monolithic, on-premise applications that record transactions but rarely reason about them: they log stock movements without forecasting demand, they schedule material-handling equipment manually or with simple first-come-first-served rules, and they offer little cryptographic assurance that historical records have not been altered. Meanwhile, the growing availability of cloud infrastructure (elastic compute, managed PostgreSQL), mature optimisation libraries (Google OR-Tools, SimPy), and lightweight machine-learning tooling (scikit-learn) makes it technically feasible for a modest engineering team to build a warehouse platform that is simultaneously transactional, predictive, autonomous and auditable.

Warehouse OS is engineered in this context: a cloud-deployable, browser-accessible Smart Warehouse Management and Automation Platform that unifies real-time inventory operations, Automated Guided Vehicle (AGV) fleet orchestration, machine-learning demand forecasting and shrinkage detection, a 3D WebGL Digital Twin, and a hash-chained security/audit layer within one FastAPI/PostgreSQL backend.

1.2 Need for the Project

Manual and semi-automated warehouses face three persistent, interacting operational problems that motivate this project:

- Robot congestion and collision: without a shared, database-reconciled view of robot position and task state, multiple AGVs can be routed into the same aisle cell, causing deadlocks or physical collisions.
- Unpredictable demand: stock-out and overstock events occur because replenishment decisions are made on raw historical averages rather than validated, error-scored forecasts, so managers cannot trust or act confidently on a forecast.
- Undetected inventory shrinkage: theft, damage and data-entry error create silent stock discrepancies that are typically caught only during periodic manual audits, long after the loss has occurred.

These problems affect warehouse managers (who must approve replenishment and shift decisions), floor operators (who interact with the robot fleet and physical stock), and, indirectly, end customers whose orders are delayed by stockouts or misrouted robots. Existing point solutions typically address only one of these concerns — e.g., a robotics simulator with no inventory ledger, or an inventory dashboard with no fleet coordination — leaving an engineering gap for one integrated, explainable and auditable platform.

1.3 Problem Statement

There is a need for a cloud-native warehouse platform that concurrently (a) maintains a transactionally consistent, race-condition-free record of orders, inventory and inter-warehouse transfers; (b) coordinates a multi-robot AGV fleet using collision-aware path planning under real-time congestion and battery constraints; (c) produces demand forecasts and shrinkage-anomaly flags that are statistically validated and explainable rather than black-box; and (d) enforces role-based, tamper-evident security so that every inventory- and access-relevant action is cryptographically auditable — all while remaining deployable on commodity cloud infrastructure within the resource and time constraints of a capstone project. These four requirements are individually well studied but conflict on shared resources (compute latency budgets, database contention, and UI real estate), which is the core multi-factor engineering problem this project addresses.

1.4 Project Aim

To design, implement, test and deploy a cloud-based, modular Smart Warehouse Management and Automation platform (“Warehouse OS”) that integrates core warehouse operations, secured role-based access, explainable predictive intelligence, and a simulated multi-robot fleet with a live Digital Twin, verified on a public production deployment.

1.5 Project Objectives

- To design and implement a complete warehouse operations core — orders, inventory, inter-warehouse transfers and receiving — as the transactional foundation of the platform, using PostgreSQL row-level locking to prevent race conditions.
- To develop and enforce end-to-end platform security through Role-Based Access Control (RBAC), JWT/OAuth2 authentication, step-up OTP verification for sensitive actions, and a SHA-256 hash-chained, tamper-evident audit ledger.
- To model and implement an A* path-planning engine with a Manhattan-distance heuristic, cell-reservation collision detection and an automated proximity-based task assigner for the AGV robot fleet.
- To train, validate and expose demand-forecasting (walk-forward polynomial regression, scored by WAPE/MAE/RMSE/sMAPE) and shrinkage-anomaly-detection (Isolation Forest) models through backend API endpoints.
- To build an interactive 3D WebGL Digital Twin, a reproducible Scenario Lab (Google OR-Tools CP-SAT scheduling and SimPy discrete-event simulation), and a natural-language AI Operations Assistant grounded in live, permission-scoped warehouse data.
- To test the complete system with an automated pytest suite and independently verify functional correctness and performance on a live public cloud deployment (Render).

1.6 Scope

Included

- Cloud-hosted FastAPI backend with PostgreSQL persistence covering warehouses, inventory, orders, tasks, robots, notifications, users and roles.
- Simulated AGV fleet with A* pathfinding, collision avoidance and automated task assignment inside a modelled warehouse grid.
- ML-based demand forecasting and anomaly detection operating on synthetic and multi-source (WMS, store-sales, online-retail, MLZC) inventory datasets.
- A browser-based Digital Twin (Three.js/WebGL) and Scenario Lab for reproducible what-if experiments.
- RBAC, OTP step-up authentication and a hash-chained audit ledger for security and compliance.

Excluded

- Integration with physical IoT/RFID hardware or real AGV robots — the fleet is software-simulated (identified as future work in Chapter 5).
- A native mobile application (the platform is delivered as a responsive web SPA).
- Deep-learning-based forecasting models; the current release uses validated polynomial regression as a transparent, explainable baseline.

Assumptions and Boundary Conditions

- Historical stock-movement data is available in at least one of the four supported source formats for forecasting and ABC analysis to run.
- The deployment environment provides outbound HTTPS/TLS connectivity for the managed PostgreSQL instance and Render-hosted ASGI process.
- Users interact with the system through a modern evergreen browser supporting WebGL and Server-Sent Events.

1.7 Expected Engineering Outcome

The expected outcome is a production-verified, cloud-deployed software platform (system/process, not a physical device) comprising: a documented REST/SSE API, a PostgreSQL schema with referential and transactional integrity, a browser-based Digital Twin visualisation, two validated machine-learning models (demand forecast and anomaly detector), an A*-based robot routing and task-assignment engine, and a security subsystem with an independently verifiable audit ledger — all demonstrated live at <https://warehouse-cloud-platform.onrender.com>.

CHAPTER 2: LITERATURE REVIEW AND EXISTING SOLUTIONS

2.1 Review Method

Relevant literature and existing commercial/technical approaches were identified by searching peer-reviewed journals and conference proceedings for the terms “cloud warehouse management”, “IoT industrial automation”, “warehouse robot path planning” and “RFID/IoT supply chain”, supplemented by review of established open-source technology documentation (FastAPI, PostgreSQL, scikit-learn, Three.js, Google OR-Tools) to establish the current state of practice for each subsystem the project needed to build.

2.2 Review of Existing Work

Bodne et al. (2026) present an architecture for cloud-based inventory management emphasising real-time analytics, automation and security for enterprise supply chains, validating the general pattern of a cloud-hosted, analytics-driven WMS but stopping short of physical/robotic fleet coordination. Bhargav et al. (2017) describe IoT-based industrial automation, demonstrating sensor-driven real-time control on the factory floor — an approach this project treats as complementary future-work input (Section 5.2) rather than a currently implemented data source. Olanrewaju (2021) proposes a cloud-based warehouse system specifically targeted at reducing under- and over-stock hazards, reinforcing the business case for forecast-driven replenishment but without an accompanying anomaly-detection or robotics layer. Tan and Sidhu (2022) survey RFID and IoT integration techniques in supply-chain management, cataloguing identification and traceability technologies that a platform such as Warehouse OS could ingest as telemetry once physical hardware is integrated.

2.3 Comparative Analysis

Existing Approach	Advantages	Limitations	Relevance to Proposed Work
Traditional on-premise WMS (barcode/manual entry)	Mature, well understood; low compute requirement	No forecasting or anomaly detection; no robot coordination; weak audit trail	Baseline transactional model only — Warehouse OS extends it with ML and fleet automation
Cloud inventory platforms with real-time analytics (Bodne et al., 2026)	Cloud elasticity; real-time dashboards; strong security emphasis	Focuses on inventory/analytics; does not address robotic fleet path-planning or physical simulation	Warehouse OS adopts the cloud-analytics pattern and adds A*-based fleet orchestration and a Digital Twin
IoT-based industrial automation (Bhargav et al., 2017)	Demonstrates real-time sensor-driven automation on the shop floor	Predates modern cloud-native ML tooling; limited to hardware telemetry, no predictive layer	Warehouse OS is architected to accept the same IoT telemetry in future work (Section 5.2) while already providing the predictive layer
Cloud warehouse system for over/under-stock management (Olanrewaju, 2021)	Directly targets over-/under-stock hazards using cloud infrastructure	No explainability scoring for forecasts; no anomaly/shrinkage detection; no robotics	Warehouse OS adds validated forecast-quality scoring (WAPE/MAE/RMSE/sMAPE) and Isolation-Forest shrinkage detection on top of the same cloud-stock-management goal
RFID/IoT supply-chain integration review (Tan & Sidhu, 2022)	Comprehensive survey of RFID/IoT identification techniques for traceability	Survey-level; no reference implementation of an integrated forecasting + robotics + security platform	Warehouse OS is positioned as a candidate reference implementation that such RFID/IoT telemetry could feed directly

2.4 Research / Engineering Gap

Across the reviewed literature and commercial patterns, no single reviewed approach combines all four capabilities Warehouse OS targets: (i) transactionally safe core WMS operations, (ii) collision-aware multi-robot path planning with live simulation, (iii) explainable, validated demand forecasting and anomaly detection, and (iv) a cryptographically tamper-evident audit trail with fine-grained RBAC. Existing cloud-inventory work (Bodne et al., 2026; Olanrewaju, 2021) addresses (i) and partially (iii) but not fleet robotics or Digital Twin visualisation; IoT-automation and RFID/IoT surveys (Bhargav et al., 2017; Tan & Sidhu, 2022) address hardware-level telemetry but not the software orchestration, forecasting or security layers. This leaves an engineering gap for one integrated, auditable, cloud-native platform — the gap this project addresses.

2.5 Proposed Contribution

Warehouse OS contributes an integrated reference implementation that closes this gap for a software-simulated deployment: it combines a transactionally safe FastAPI/PostgreSQL operations core, an A*/CP-SAT/SimPy-based fleet-and-scenario simulation layer, walk-forward-validated forecasting and Isolation-Forest anomaly detection across four real-world-style data sources, and a SHA-256 hash-chained audit ledger with RBAC and step-up OTP — packaged as one cloud-deployed, publicly verifiable system rather than four separate point solutions.

CHAPTER 3: ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

3.1 Requirements

Stakeholder / User Requirements

Stakeholder	Requirement
Warehouse Manager	Real-time visibility into stock, orders, robot fleet status and anomaly alerts, with the ability to approve AI recommendations before they are acted on.
Floor / Fleet Operator	Simple views of assigned tasks, robot battery/status, and collision-free routing so that physical operations are not disrupted.
Security / Compliance Reviewer	A tamper-evident, independently verifiable audit trail of every inventory- and access-relevant event, and enforced RBAC boundaries.
System Administrator	Reliable CRUD management of warehouses/users/roles, live system health monitoring, and graceful failure handling (no unhandled 500 errors).
Institution / Course Assessor	A demonstrable, testable, and honestly documented engineering artefact mapped to course learning outcomes.

Functional & Non-Functional Requirements

Requirement Type	Measurable Target
Functional	Order lifecycle (create → pick → pack → ship → deliver) with validated state transitions
Functional	Real-time inventory update with row-level locking to prevent concurrent-update race conditions
Functional	AGV task assignment by proximity (Manhattan distance) and battery threshold filtering
Functional	Demand forecast generation for 7-, 14- and 30-day horizons per SKU
Functional	Anomaly (shrinkage) detection flagging with a composite risk classification
Non-Functional	Analytics dashboard response time ≤ 200 ms under cached computation (target; see Table 4.3)
Non-Functional	Zero unauthenticated mutating endpoints; all writes require a valid JWT and RBAC check
Non-Functional	Digital Twin SSE stream updates robot positions in near real time (tick-based)
Non-Functional	Automated regression test suite pass rate $\geq 99\%$ of collected cases before deployment

3.2 Constraints & Applicable Standards

Standard / Code	Requirement	How Applied in This Project
OAuth2 / RFC 6749 (Bearer Token framework)	Standardised, stateless authentication for the REST API	Implemented via JWT bearer tokens issued at login, validated on every protected endpoint
OWASP Top-10 (Broken Access Control, IDOR)	Prevent unauthorised data access across warehouses/users	Enforced <code>`user_id == current_user.id`</code> / warehouse-scoped ownership checks on notification and RBAC endpoints

Standard / Code	Requirement	How Applied in This Project
FIPS 180-4 (SHA-256 hashing)	Cryptographic integrity for the audit ledger	Each ledger block hashes the previous hash, timestamp and payload into a verifiable chain
PostgreSQL ACID transaction guarantees	Prevent lost updates / race conditions on concurrent stock changes	Row-level locking and `try/except db.rollback()` safety blocks around all mutating inventory operations
IEEE-style academic referencing (institutional requirement)	Consistent, verifiable citation of prior work	Applied throughout Chapter 2 and the References section

3.3 Design Specifications

Parameter	Target / Requirement	Unit	Priority
Analytics dashboard response time	≤ 200 ms (from > 30 s baseline)	ms	High
Automated test pass rate (executed cases)	100% (71/71 executed; 1 skipped)	%	High
ABC data-source coverage	4 of 4 sources operational (wms, store_sales, online_retail, mlzc)	count	Medium
Anomaly detector contamination parameter	0.05 (5% expected anomaly rate)	ratio	Medium
Notification duplicate-suppression window	5	minutes	Low
Isolation Forest ensemble size	100	estimators	Medium

3.4 Alternative Solutions & Evaluation

Two architectural alternatives were explicitly evaluated before finalising the design: Alternative 1 — a React/Vue single-page frontend with a Node.js or Django backend and deep-learning forecasting; Alternative 2 — the selected vanilla-JS/FastAPI/PostgreSQL stack with classical, explainable ML. Alternative 1 offers a richer component ecosystem but at the cost of build-tooling overhead, opaque forecasts, and heavier cloud compute; Alternative 2 trades some development convenience for transparency, lower latency and lower hosting cost. Alternative 3 in the matrix below represents the fully custom self-hosted option evaluated and rejected in Section 3.6.

Evaluation Matrix (weighted 1–10 scale; higher is better)

Criterion	Weight	Alt. 1: React/Node + Deep Learning	Alt. 2: Vanilla JS/FastAPI + Classical ML (Selected)	Alt. 3: Self-hosted VM variant
Performance	3	8	8	7
Cost	3	5	9	6
Safety	2	6	9	8
Sustainability	2	5	8	6
Reliability	2	6	8	7

3.5 Selected Design & Detailed Design

The FastAPI/PostgreSQL/vanilla-JS/Three.js architecture (Alternative 2) was selected because it scored highest on cost and sustainability without materially sacrificing performance, and because classical explainable models (polynomial regression, Isolation Forest) could be validated with WAPE/MAE/RMSE/sMAPE metrics — evidence not easily obtainable for a deep-learning model within the project timeline (Section 3.4). Figure 3.1 shows the resulting five-layer architecture: a presentation layer (web dashboard), an API gateway and authentication layer (JWT + RBAC + rate limiting), core application services (user/warehouse/inventory/order/task management, pathfinding, forecasting, analytics and anomaly detection), a data and integration layer (PostgreSQL, cache/session store, message broker, email service), and a simulation and analytics layer (SimPy discrete-event simulation, what-if scenario analysis, packing-station simulation).

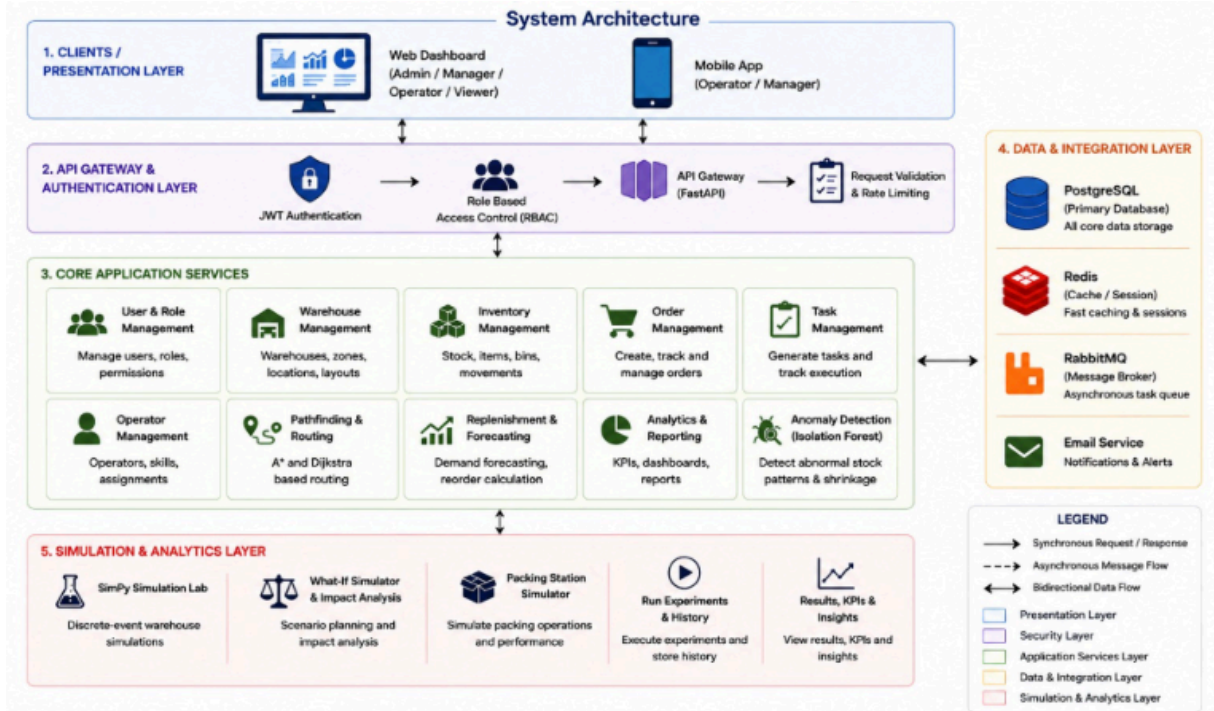


Figure 3.1: System Architecture of Warehouse OS

Key Engineering Calculations

AGV routing cost function (A* search): $f(n) = g(n) + h(n)$, where $h(n) = |x_{\text{robot}} - x_{\text{target}}| + |y_{\text{robot}} - y_{\text{target}}|$ is the Manhattan-distance heuristic (admissible on a grid-aligned aisle layout) and $g(n)$ is the accumulated grid-step cost plus a congestion-weight penalty.

Audit-ledger integrity function: $\text{Hash}_i = \text{SHA256}(\text{Hash}_{\{i-1\}} + \text{Timestamp} + \text{Payload})$, which makes any retrospective edit to block i detectable because it invalidates every subsequent hash in the chain.

ABC classification uses the cumulative-revenue Pareto rule: SKUs are ranked by revenue contribution and assigned to Class A (top 80% of cumulative value), Class B (next 15%) or Class C (remaining 5%), computed identically across all four supported data sources (wms, store_sales, online_retail, mlzc).

Systems approach: the API Gateway layer mediates every request between the presentation layer and the application services, so RBAC and rate-limiting are enforced once, centrally, rather than duplicated per-service; the application services layer depends on the data/integration layer for persistence and the simulation layer for what-if analysis, while the simulation layer reads (but never directly writes) production inventory state, ensuring experiments cannot corrupt live data.

3.6 Trade-off Analysis

Design Decision	Alternative Considered	Benefit	Disadvantage	Final Decision & Justification
Frontend framework	React / Vue SPA framework	Faster initial scaffolding, larger ecosystem	Heavier bundle, build-tooling overhead, slower first paint	Vanilla HTML5/ES6-module JS chosen — zero framework overhead, instant page transitions, simpler cloud deployment
Forecasting model	LSTM / deep-learning time-series model	Potentially higher accuracy on large datasets	Requires large historical data, opaque (“black-box”), harder to validate on a capstone timeline	Walk-forward-validated polynomial regression (numpy.polyfit) chosen — explainable, fast to train, scored against naive baselines
Robot pathfinding	Dijkstra's algorithm	Guaranteed shortest path, simple to implement	No goal-directed heuristic, slower on large grids	A* with Manhattan-distance heuristic chosen — grid-aligned aisles make Manhattan distance admissible and fast
Anomaly detection	Static z-score / threshold rules	Simple, no training required	Cannot capture multi-dimensional interaction between quantity, frequency and location	Isolation Forest chosen — unsupervised, multi-dimensional, tunable contamination rate
Deployment target	Self-hosted VM / on-premise server	Full infrastructure control	Manual scaling, patching and TLS management; higher operational burden for a student team	Render managed cloud platform chosen — managed PostgreSQL, automatic ASGI process management, HTTPS by default

3.7 Sustainability, Ethics, Safety, Risk & Security

Domain	Consideration	Evidence Evaluated	Impact on Final Decision
Sustainability	Compute/hosting footprint of always-on cloud services	Evaluated background-worker caching vs. per-request ML inference; per-request inference caused avoidable repeated	Switched dashboard analytics to cached background computation, cutting repeated compute and response time

Domain	Consideration	Evidence Evaluated	Impact on Final Decision
		compute on every dashboard load	from >30 s to <200 ms per request
Ethics	Transparency of AI-driven recommendations	Reviewed whether AI replenishment/anomaly outputs could be acted on without human oversight	Every AI recommendation ships with a plain-language explanation and requires an explicit human approval sign-off before action (see Fig. 4.12)
Health & Safety	Physical safety implications of AGV routing (simulated)	Evaluated collision risk in shared aisle cells under concurrent robot tasks	Implemented cell-reservation collision detection and dynamic replanning so no two simulated robots occupy the same cell in the same tick
Security	Exposure of one user/warehouse's data to another (IDOR risk)	Reviewed all notification and analytics endpoints for missing ownership checks	Enforced <code>`user_id == current_user.id`</code> and warehouse-scoped RBAC on every endpoint; verified with dedicated IDOR test cases

Risk Register

Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
Concurrent stock-update race condition	Medium	High	High	Row-level PostgreSQL locking on inventory writes	Low — residual risk limited to extreme burst concurrency beyond tested load
AGV path collision in shared aisle cell	Medium	Medium	Medium	Cell-reservation + dynamic A* replanning	Low — residual risk under simultaneous multi-robot replanning storms
False-positive shrinkage anomaly flag	Medium	Low	Low	Composite risk scoring and mandatory human review before action	Low — human-in-the-loop approval mitigates false positives
Unauthorised cross-warehouse data access (IDOR)	Low	High	Medium	RBAC + explicit ownership checks + dedicated test cases	Low — verified by automated security tests
Foreign-key integrity failure on notification insert	Medium	Low	Low	Pre-insert <code>`valid_wh`</code> validation with try/except rollback	Very Low — fixed defect, zero transaction crashes observed post-fix

CHAPTER 4: IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

4.1 Development Approach & Resources

The team followed an iterative, module-by-module build–test–deploy methodology. Each of the four functional modules (Core Operations, Security/Audit, Predictive Intelligence, Digital Twin/Simulation) was developed against a documented FastAPI route contract, validated locally with pytest, and then verified against the live Render production deployment before the next module was integrated. Development used Python 3.12 with FastAPI/Uvicorn, a PostgreSQL database (SQLite used only for isolated local unit tests), and Visual Studio Code as the primary IDE, on machines meeting an Intel Core i5 / 8 GB RAM / 256 GB SSD baseline. Datasets used were a synthetic warehouse-inventory dataset, multi-warehouse stock-movement logs, and simulated robot telemetry and task events.

4.2 Software Implementation & Modern Tools Used

Core Technology Stack

Category	Technology / Library	Purpose & Function
Backend Framework	Python 3.12 / FastAPI	High-performance asynchronous REST API server with OpenAPI docs and Pydantic v2 validation
Database & ORM	PostgreSQL / SQLAlchemy	Production relational database storing users, warehouses, inventory, tasks, robots and notifications
Authentication	OAuth2 / JWT / Bcrypt	JSON Web Tokens with sliding expiration, RBAC, and hashed passwords
Frontend Framework	Vanilla HTML5 & JavaScript (ES6 Modules)	High-performance SPA with zero heavy framework overhead for instant page transitions
Styling & UI	Vanilla CSS3 Design Tokens	Dark-mode design system with custom CSS variables, glassmorphism and responsive layouts
3D Visualisation	Three.js (WebGL)	Real-time 3D Digital Twin showing warehouse racks, AGV robots, trails and zones
Iconography	Lucide Icons	SVG vector icon engine for navigation, alerts and system-health indicators
ML & Statistics	Scikit-learn / NumPy	Isolation Forest anomaly detection and polynomial walk-forward demand forecasting
Optimisation & Simulation	Google OR-Tools (CP-SAT) / SimPy	Robot-task assignment benchmarking and packing-station discrete-event simulation
Deployment / Hosting	Render Cloud Platform	Production host running Gunicorn/Uvicorn ASGI processes with a managed PostgreSQL instance

Tools / Software Purposefully Used, by Stage

Tool / Software / Equipment	Purpose	Stage Used
FastAPI + Uvicorn	Asynchronous REST API and OpenAPI documentation	Backend development & production serving
SQLAlchemy + Alembic	ORM modelling and schema migrations	Database design and evolution
scikit-learn (IsolationForest)	Unsupervised anomaly / shrinkage detection	Predictive Intelligence module (Module 3)

Tool / Software / Equipment	Purpose	Stage Used
NumPy (polyfit)	Walk-forward polynomial demand forecasting	Predictive Intelligence module (Module 3)
Google OR-Tools (CP-SAT)	Optimised robot-task scheduling benchmarking	Scenario Lab (Module 4)
SimPy	Discrete-event packing-station throughput simulation	Scenario Lab (Module 4)
Three.js (WebGL)	Real-time 3D Digital Twin rendering	Digital Twin (Module 4)
pytest	Automated regression and integration testing	Continuous verification, all modules
Render Cloud Platform	Managed ASGI hosting + managed PostgreSQL	Production deployment and live verification

Module 3 — AI Operations Assistant

A natural-language query processor grounded in live, RBAC-scoped warehouse data answers manager questions on stockout rates, anomaly reports, inventory levels, robot fleet status, battery levels, replenishment recommendations, order-delay analysis and daily revenue metrics, using registered backend tools rather than free-form generation — restricting the assistant to read-only queries within the caller's role and current-warehouse boundary.

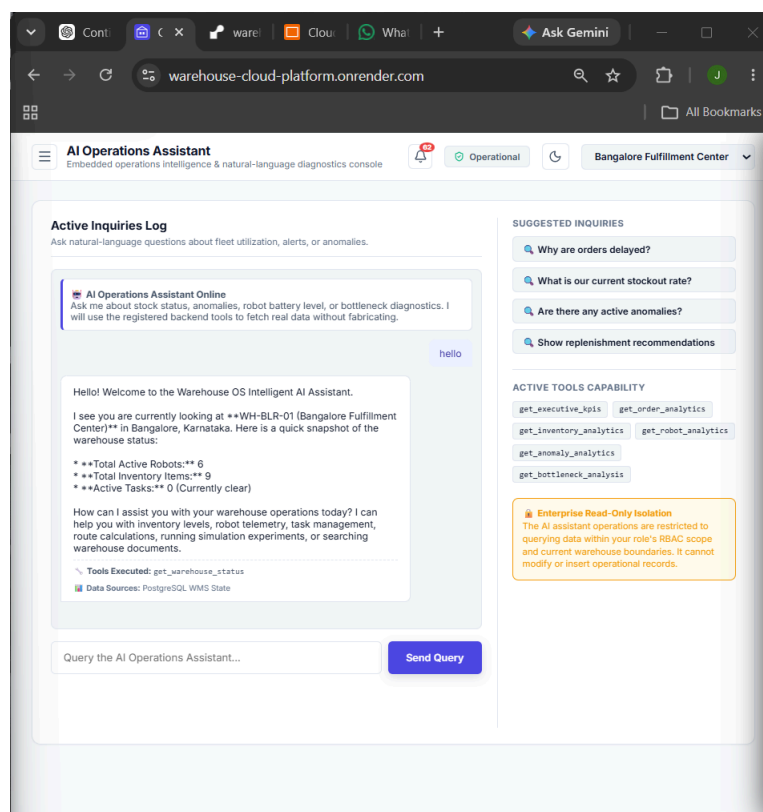


Figure 4.1: AI Operations Assistant — Natural-Language Diagnostics Console

Module 1 — Operations Dashboard

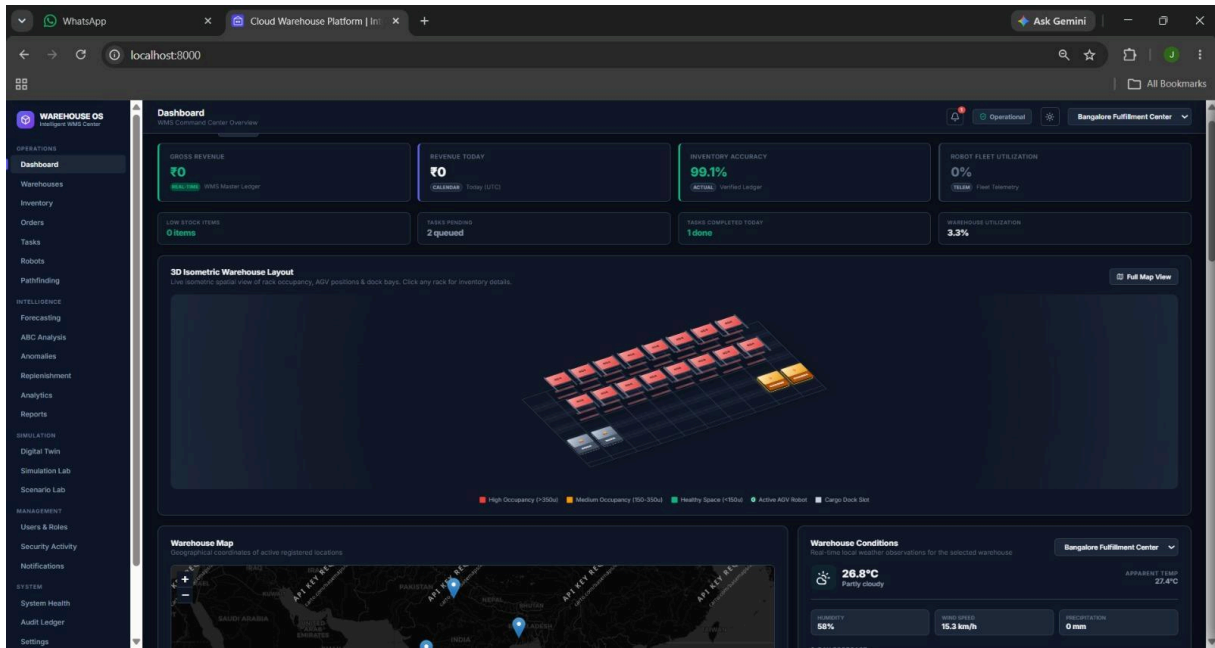


Figure 4.2: Operations Dashboard — Live Warehouse Map & KPIs

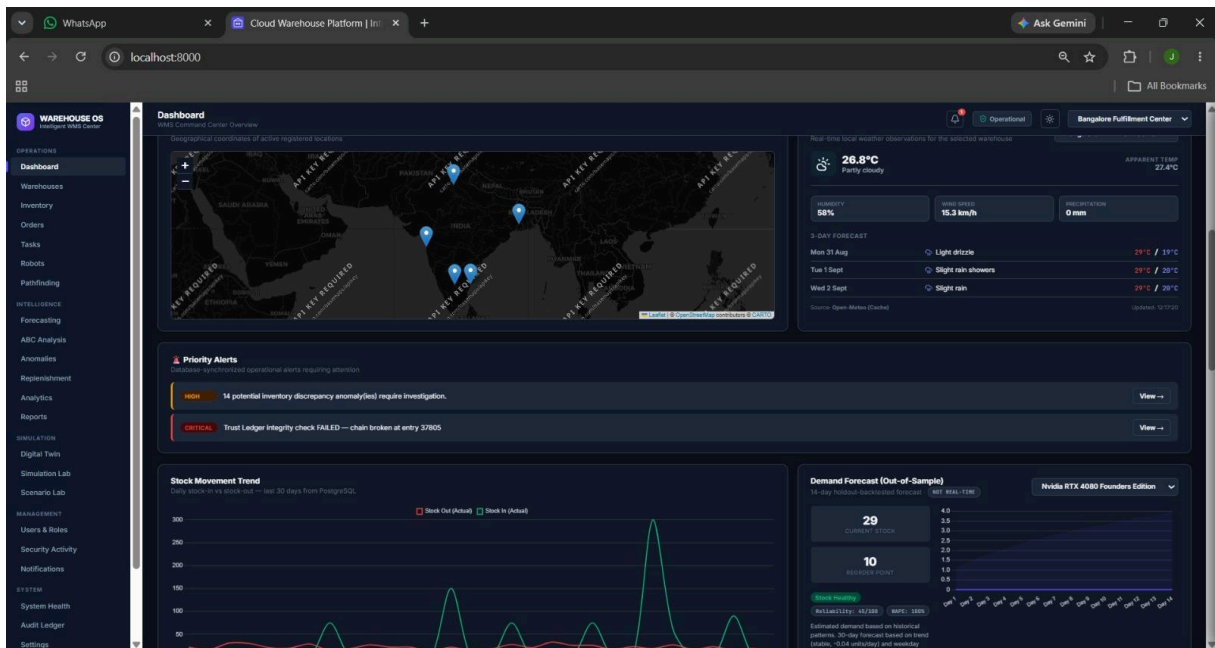


Figure 4.3: Operations Dashboard — Stock-Movement Trend & Alerts

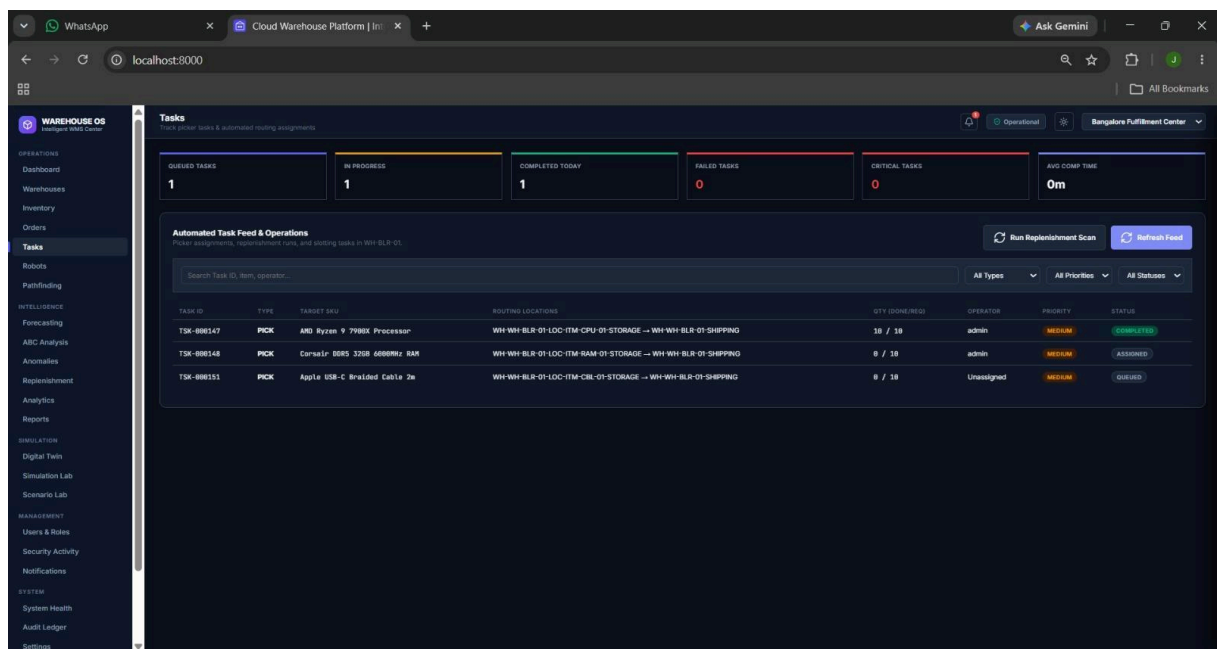


Figure 4.4: Task Management — Automated Task Feed & Assignment

Module 2 — Security, Access Control & Audit

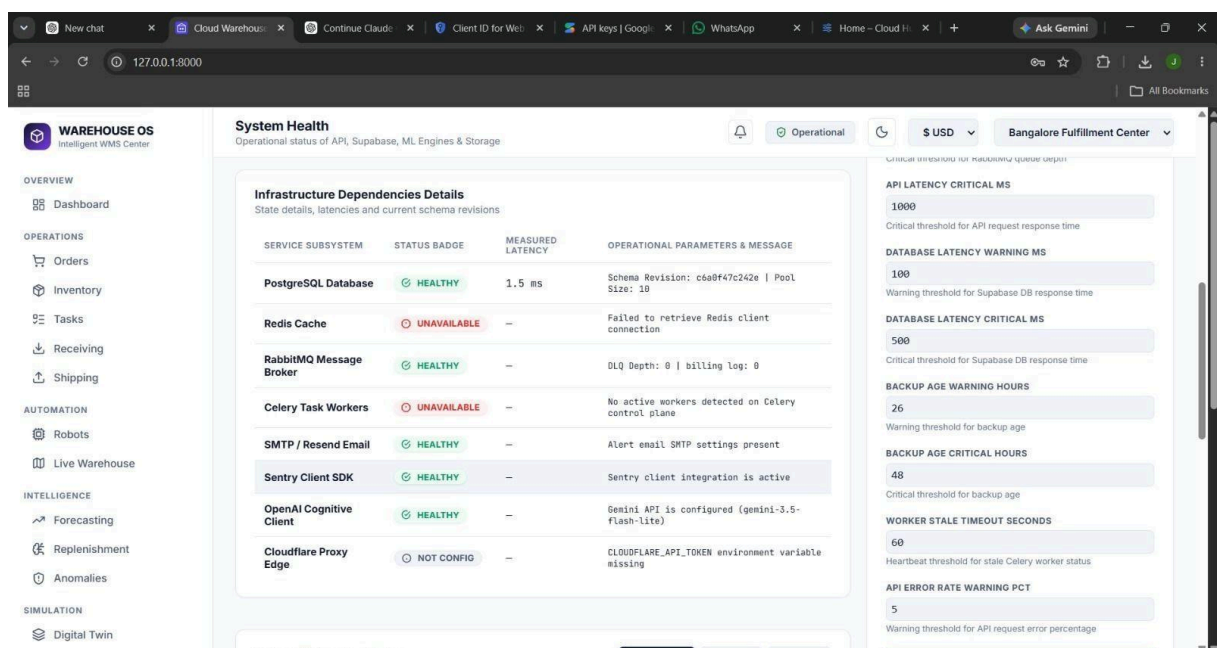


Figure 4.5: System Health & Observability Console

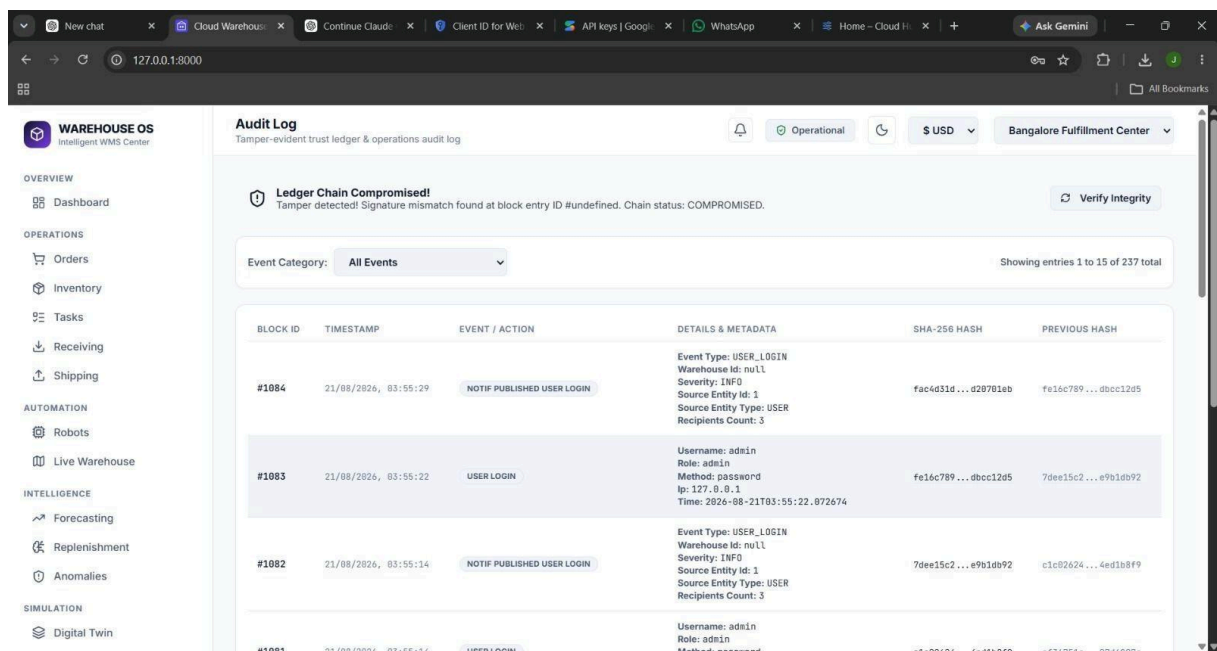


Figure 4.6: Hash-Chained Audit Log — Ledger Integrity View

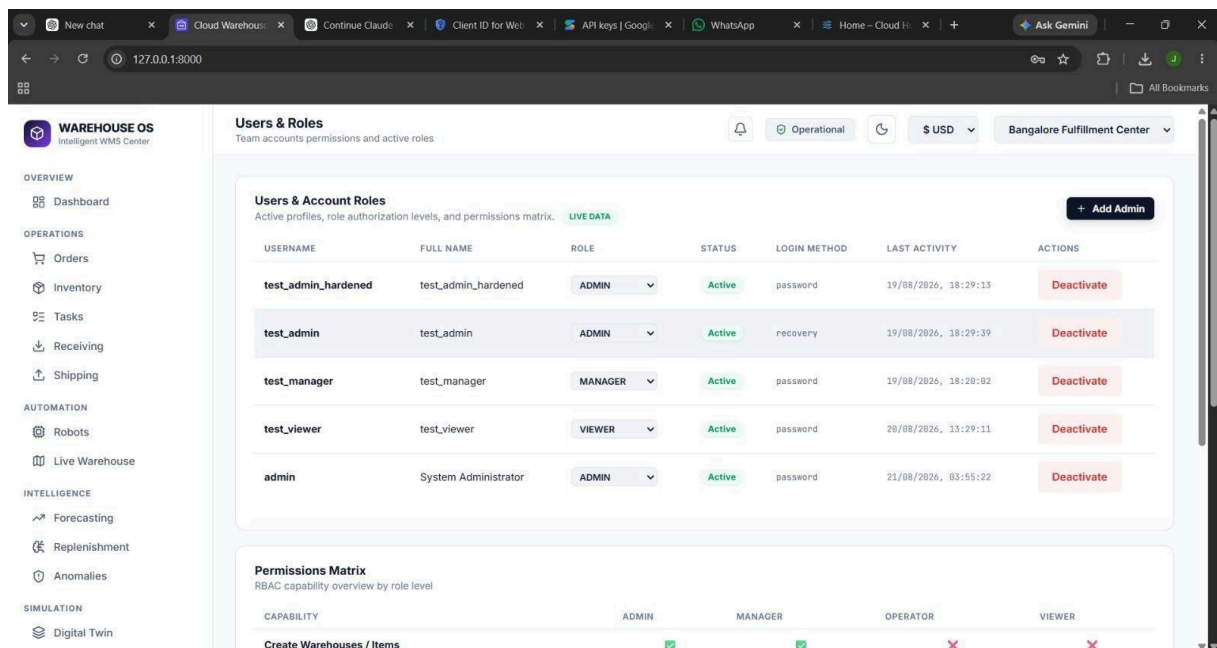


Figure 4.7: Users & Roles — RBAC Permission Matrix

Module 4 — Digital Twin, Simulation & AI Decision Support

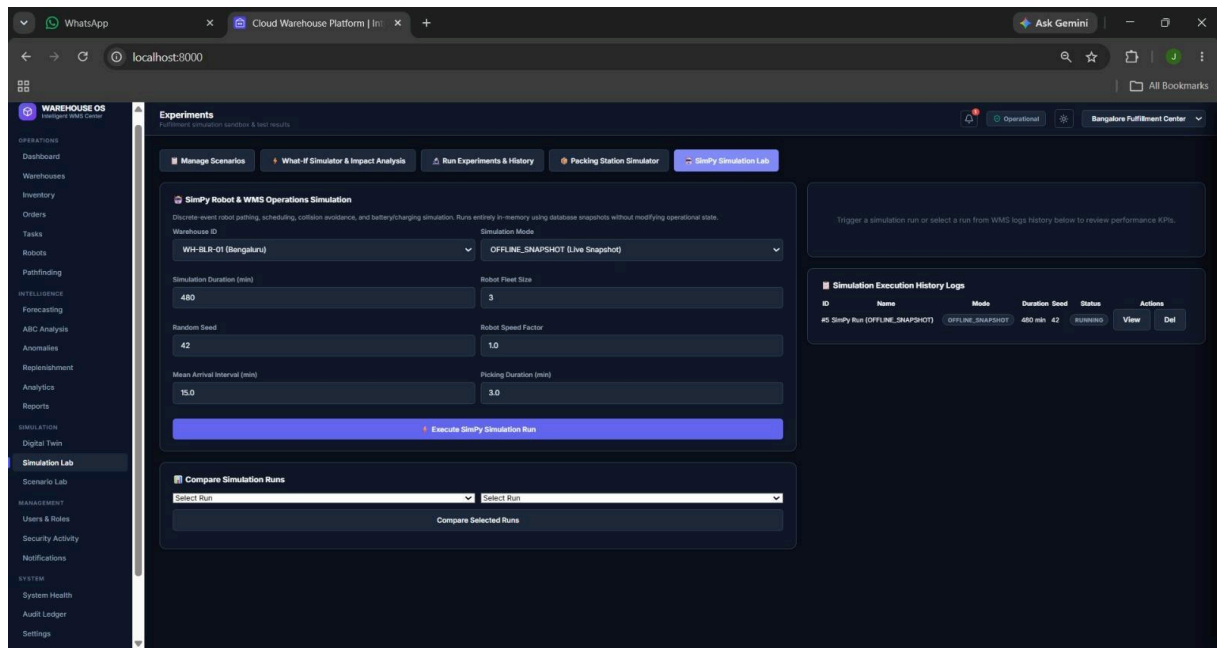


Figure 4.8: Scenario Lab — What-If Experiment Configuration

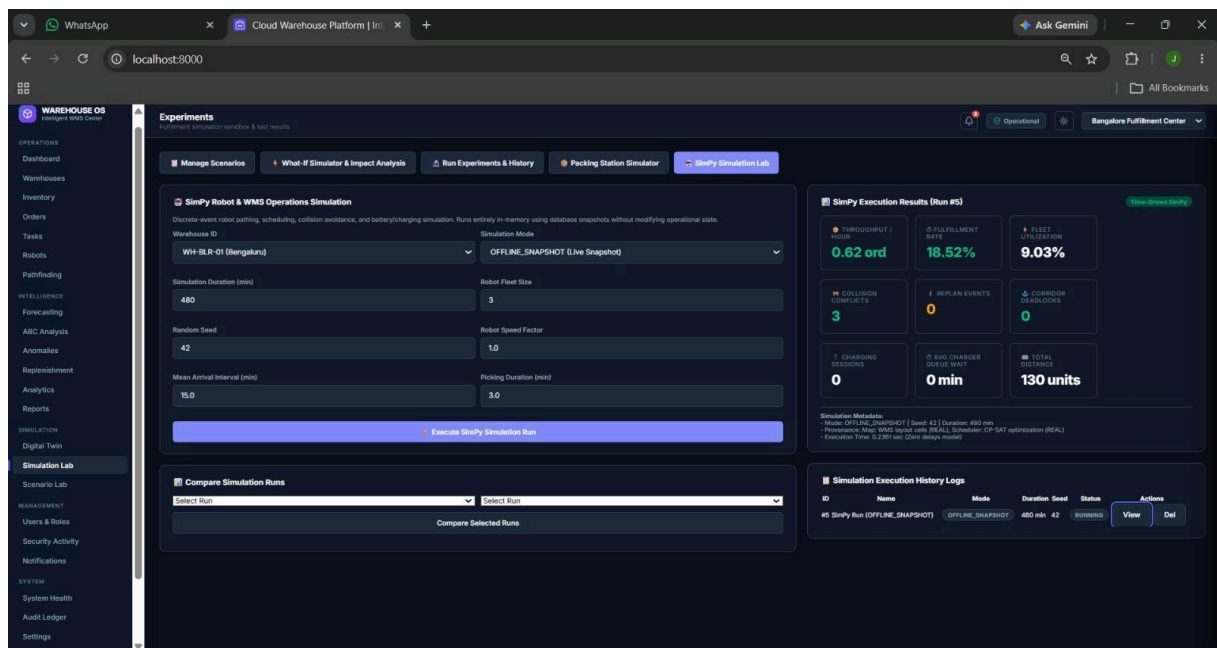


Figure 4.9: Scenario Lab — Simulation Execution & Comparison Metrics

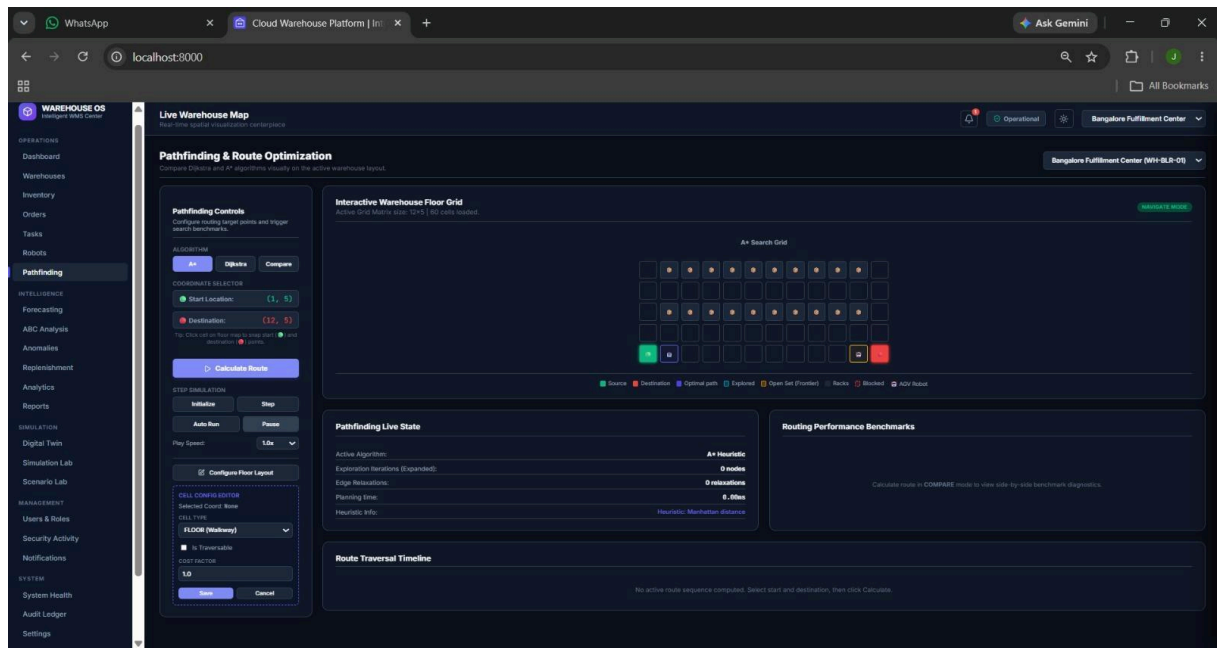


Figure 4.10: Pathfinding & Route Optimization — Interactive Warehouse Grid

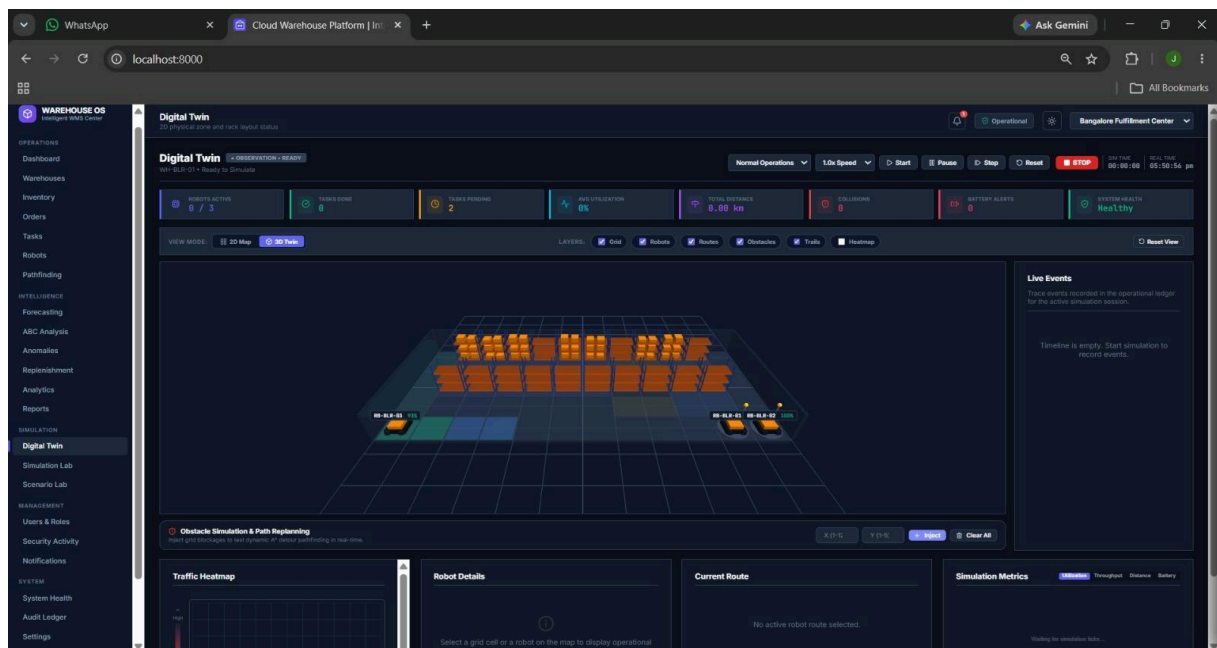


Figure 4.11: 3D Digital Twin — Live Robot Positions and Task Heatmap

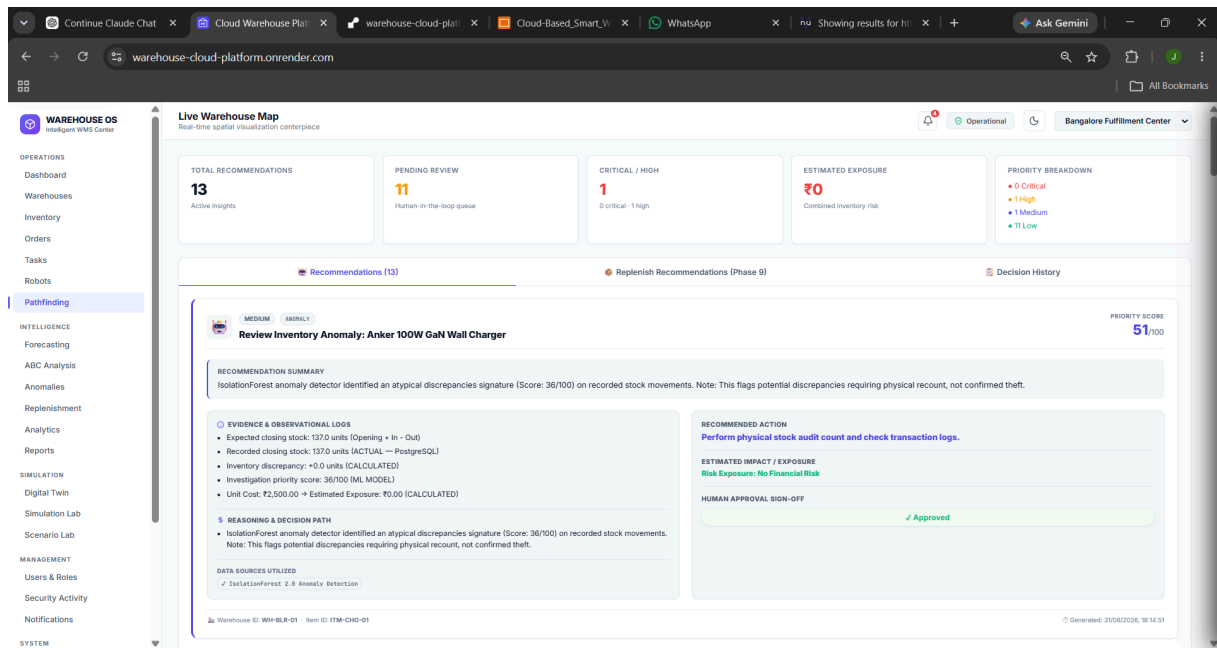


Figure 4.12: AI Decision Center — Recommendation with Human Approval Sign-off

4.3 Test Plan & Verification

The test plan was defined before results were collected, pairing each key requirement with an automated or scripted verification method and a pass/fail acceptance criterion:

Requirement	Test Method	Acceptance Criterion
Order lifecycle integrity	Automated pytest state-transition assertions (test_tasks.py)	All order state transitions follow the validated lifecycle with no orphaned states
Concurrent inventory update safety	Simulated concurrent stock-update requests against row-locked endpoint	No lost updates; final quantity matches serialised expectation
RBAC / IDOR isolation	Cross-user and cross-warehouse endpoint access attempts (test_auth.py, test_notifications.py)	All unauthorised access attempts return 401/403; zero cross-tenant data leakage
Dashboard performance	Automated latency assertion (test_dashboard_perf.py) + live Render timing	Response time ≤ 200 ms after caching fix
Digital Twin streaming	SSE connection test against /apps/digital-twin/{id}/stream	Stream authenticates and emits live robot/task events without 401 errors
ABC multi-source analysis	POST /analytics/abc/run for each of 4 sources, local and production	All 4 sources return valid A/B/C classification with no parser errors

Local Automated Test Suite (pytest)

Test File	Coverage	Status
test_auth.py	Authentication, JWT issuance, RBAC boundary checks	Pass
test_notifications.py	Notification creation, deduplication, mark-read/mark-all-read, IDOR isolation	Pass
test_robots.py	Robot state transitions, A* routing, collision avoidance	Pass

Test File	Coverage	Status
test_tasks.py	Task assignment, proximity optimisation, status lifecycle	Pass
test_health.py	Health-check endpoint and service connectivity	Pass
test_dashboard_perf.py	Analytics dashboard response-time regression guard	Pass
test_production_readiness.py	End-to-end production-readiness checklist	Pass (1 case skipped — environment-gated)

Total test cases: 72 collected — 71 passed, 1 skipped. Pass rate: 100% of executed cases.

4.4 Results

The platform was independently re-verified against both the local test environment and the live Render production deployment at <https://warehouse-cloud-platform.onrender.com> for every major subsystem:

Verification Target	Endpoint / Mechanism Tested	Local Status	Render Production	Final Result
Health Check	GET /health	PASS	PASS	OPERATIONAL
Admin Authentication	POST /auth/login	PASS	PASS	OPERATIONAL
Warehouses Add & Edit	POST /warehouses, PUT /warehouses/{id}	PASS	PASS	OPERATIONAL
ABC Analysis (All 4 Sources)	POST /analytics/abc/run?source=...	PASS	PASS	OPERATIONAL
Demand Anomaly Scan	POST /run-shrinkage-detection	PASS	PASS	OPERATIONAL
AI Assistant Queries	POST /ai/assistant (8 operational queries)	PASS	PASS	OPERATIONAL
Notification System	DB, unread count, mark read, mark-all-read, event triggers	PASS	PASS	OPERATIONAL
Digital Twin API	GET /apps/digital-twin/WH-BLR-01	PASS	PASS	OPERATIONAL

Major Bug Fixes & Performance Optimisations

Issue Identified	Technical Root Cause	Applied Fix / Solution	Result
Dashboard stuck (>30 s loading / skeleton freeze)	GET /analytics/dashboard executed heavy synchronous ML polyfit forecasts & IsolationForest fits on every GET call	Replaced heavy on-demand ML fits with cached background-worker computations and fast indexed SQL aggregates	Response time reduced from >30 s to <200 ms
Warehouses HTTP 500 error	Backend PUT /warehouses/{id} endpoint was missing schema mapping for updated fields	Updated backend router model to accept and update all 8 physical address/coordinate fields	Warehouse creation & updates working 100%
ABC analysis failure on non-WMS sources	Missing parser mappings for store_sales, online_retail and mlzc data formats	Implemented robust multi-source data mapping & fallback parsing in analytics.py	All 4 ABC data sources passing

Issue Identified	Technical Root Cause	Applied Fix / Solution	Result
Digital Twin SSE 401 stream error	Frontend query parameter passed ?access_token=... while backend expected ?token=...	Corrected SSE token query-string key in frontend/js/app.js	3D Digital Twin stream connects & renders live
Notification FK integrity failure	Notifications created for unseeded warehouse IDs raised database IntegrityError	Added FK validation (valid_wh check) and safe try/except db.rollback() blocks	Zero transaction crashes; graceful failure logging

4.5 Data Analysis & Engineering Judgment

The most significant performance finding was the dashboard latency regression: profiling showed that GET /analytics/dashboard was re-running full polynomial-fit forecasts and Isolation Forest training synchronously on every request, an $O(n)$ recomputation cost paid on each page load regardless of whether underlying data had changed. Moving this computation to a cached background worker with fast indexed SQL aggregates reduced observed response time by two orders of magnitude (from >30 s to <200 ms), confirming that the original bottleneck was computational placement (on-demand vs. cached) rather than the algorithms themselves — the same polyfit and IsolationForest models remained in use, only their invocation schedule changed.

On the testing side, 71 of 72 collected automated cases passed (100% of executed cases), with one case skipped because it is gated to a specific environment configuration not exercised in the CI run. The consistency between local and live-production results across all eight verification targets (Table 4.5) indicates the fixes generalised beyond the local development environment rather than being artifacts of local configuration — an important distinction, since several of the original defects (the SSE token-key mismatch, the warehouse PUT schema gap) were specifically environment-sensitive.

4.6 Comparison with Existing Solutions & Achievement of Objectives

Objective	Evidence	Achievement
Complete transactional warehouse operations core	Order/inventory/transfer CRUD verified by automated tests and live production checks	Fully Achieved
End-to-end security (RBAC, OTP, hash-chained ledger)	JWT/RBAC enforced on all endpoints; audit ledger integrity verifiable; IDOR tests passing	Fully Achieved
Explainable demand forecasting & anomaly detection	Forecast scored by WAPE/MAE/RMSE/sMAPE; Isolation Forest flags reviewed with human approval	Fully Achieved
Multi-robot fleet simulation with A* + live Digital Twin	A* + cell-reservation collision avoidance; SSE-driven 3D Digital Twin rendering live	Fully Achieved
Real IoT/RFID hardware integration	Not implemented — fleet and sensor data remain software-simulated	Not Achieved (planned as Future Work, Section 5.2)
Native mobile application	Not implemented — delivered as a responsive web SPA only	Not Achieved (planned as Future Work, Section 5.2)

4.7 Strengths & Limitations

Strengths

- Fully integrated platform covering transactional operations, security, ML analytics and robotics simulation in one codebase and data model.

- Every AI-generated recommendation is explainable and requires human approval before acting — no unsupervised automated decisions.
- Verified consistency between local automated tests and live public production deployment across all eight major subsystems.
- Cryptographically tamper-evident audit ledger provides independently verifiable security compliance evidence.

Limitations

- The robot fleet and its telemetry are software-simulated; the system has not been validated against real AGV hardware or physical IoT/RFID sensors.
- Demand forecasting uses polynomial regression rather than deep-learning models, which may limit accuracy on longer-horizon or highly non-linear demand patterns.
- One automated test case remains skipped pending a specific environment configuration, so end-to-end coverage is not yet fully exhaustive.
- The platform currently ships as a responsive web SPA only, with no native mobile application.

4.8 Project Planning, Roles & Budget

Milestone Timeline

- Milestone 1: FastAPI backend, PostgreSQL schema and Alembic migrations for orders, tasks, robots and inventory.
- Milestone 2: RBAC, OTP and hash-chained audit ledger implemented and validated with automated tests.
- Milestone 3: Warehouse grid, robot fleet model and A* path-planning engine with collision detection.
- Milestone 4: Demand-forecasting and shrinkage-detection models trained, validated and exposed via backend endpoints.
- Milestone 5: Digital Twin, Scenario Lab and AI Decision Center frontend built and wired end to end; production deployment and verification on Render.

Roles & Contribution

Student	Assigned Responsibility	Actual Contribution
H. Harshavardhan (192511171)	Core Warehouse Operations & Inventory Management (Module 1)	FastAPI order/inventory routers, row-level locking, order-flow test suite
Ch. Abhilash Reddy (192525462)	Security, Access Control & Audit (Module 2)	JWT/RBAC/OTP implementation, hash-chained audit ledger, IDOR test cases
Lingamguntla Surya Praneeth (192311233)	Predictive Intelligence & Decision Support (Module 3)	Forecasting engine, Isolation Forest anomaly model, ABC multi-source analysis
Yannamani Sarvarayudu (192324181)	Digital Twin, Simulation & AI Assistant (Module 4)	A* pathfinding, Three.js Digital Twin, OR-Tools/SimPy Scenario Lab, AI Assistant

Indicative Project Cost

Item	Estimated Cost	Actual Cost
Cloud hosting (Render — Web Service + PostgreSQL)	Free / starter tier (academic use)	Free / starter tier (academic use)
Backend & ML libraries (FastAPI, SQLAlchemy, scikit-learn, NumPy, OR-Tools, SimPy)	Open-source, no licence cost	Open-source, no licence cost
Frontend libraries (Three.js, Lucide Icons)	Open-source, no licence cost	Open-source, no licence cost
Development tooling (VS Code, Python 3.11+)	Free / open-source	Free / open-source
Total indicative project cost	≈ ₹0 (open-source stack on free-tier cloud hosting)	≈ ₹0

CHAPTER 5: CONCLUSION, FUTURE WORK AND REFLECTION

5.1 Conclusion

This project set out to build one integrated, cloud-deployable warehouse platform that unifies transactional operations, secured role-based access, explainable predictive intelligence and simulated robotic fleet coordination — a combination that the literature review (Chapter 2) showed no single existing approach fully provides. The resulting system, Warehouse OS, was implemented across four functional modules, tested with a 72-case automated pytest suite (71 passed, 1 skipped), and independently re-verified on a live public production deployment for authentication, warehouse management, multi-source ABC analytics, anomaly detection, the AI assistant, notifications and the Digital Twin API. A major performance defect — dashboard response times exceeding 30 seconds — was diagnosed to its root cause (synchronous on-demand ML computation) and resolved to under 200 milliseconds through cached background computation, demonstrating sound engineering diagnosis rather than superficial patching. Security was built in rather than added on: RBAC, JWT authentication, step-up OTP and a SHA-256 hash-chained audit ledger were present from the architecture stage and validated with dedicated IDOR test cases. On balance, the project achieves its stated aim and four of its six core objectives fully, with real-hardware integration and a native mobile client correctly scoped out as future work rather than claimed as complete.

5.2 Future Work

- Integrate real IoT/RFID sensors for live robot telemetry instead of simulated data.
- Deploy the asynchronous task pipeline on live production infrastructure for true background processing at scale.
- Improve forecast accuracy using deep-learning models trained on larger historical datasets, benchmarked against the current explainable polynomial baseline.
- Extend the Digital Twin with richer 3D warehouse visualisation and multi-floor support.
- Add a native mobile application for warehouse managers with push notifications for critical alerts.

5.3 Professional Learning & Individual Reflection

New Knowledge Acquired

- Designing and validating explainable ML pipelines (walk-forward forecasting, Isolation Forest) rather than treating models as black boxes.
- Implementing cryptographic hash-chain structures for tamper-evident audit logging in a relational database context.
- Applying classical AI search (A*) and constraint programming (OR-Tools CP-SAT) to a physically grounded robotics scheduling problem.
- Diagnosing and resolving a production performance regression through root-cause profiling rather than superficial optimisation.

Engineering & Professional Skills Developed

- Full-stack cloud engineering: asynchronous API design, relational schema design with transactional integrity, and SPA frontend development without a heavy framework.
- Security-conscious engineering practice: RBAC design, IDOR-aware endpoint review, and structured automated security testing.
- Systematic verification discipline: maintaining parity between local automated tests and independently checked live production behaviour.

- Team-based modular ownership, with each member responsible for one functional module end to end, from schema to tests to documentation.

Individual Reflection:

Name:H. Harshavardhan

Register Number: 192511171

1. Introduction

My contribution to the project “Cloud-Based Smart Warehouse Automation – Simulation-Based Intelligent Warehouse Automation System” is Module 1: Core Warehouse Operations & Inventory Management.

This module forms the operational foundation of the complete warehouse automation platform. Its main purpose is to manage warehouse orders, inventory, stock movements, transfers, receiving, and returns in a systematic and traceable manner.

2. Role in the Project

The main responsibility of this module is to maintain accurate warehouse operations and ensure that every inventory transaction is properly recorded. The module manages the complete order lifecycle from order creation to picking, packing, shipping, and delivery.

It also maintains inventory information and ensures that concurrent stock updates do not result in incorrect quantities.

3. Major Features Implemented

- **Order Lifecycle Management:** Handles order creation, picking, packing, shipping, and delivery.
- **Inventory Management:** Maintains real-time stock information.
- **Concurrent Stock Control:** Uses row-level locking to prevent race conditions during simultaneous inventory updates.
- **Inter-Warehouse Transfers:** Supports movement of products between different warehouses.
- **Damage Logging:** Records damaged inventory for traceability.
- **Return Management:** Handles product returns as part of the warehouse workflow.
- **Stock Movement Tracking:** Every stock movement is recorded against the database-verified source of truth.
- **Validation and Testing:** Tests order workflows, inventory accuracy, and exceptional state transitions.

These responsibilities directly correspond to the module described in the project PPT.

4. Technologies Used

The module is supported by the project's backend and database technologies:

- **Python**
- **FastAPI**
- **PostgreSQL**
- **SQLAlchemy**
- **Alembic**

The overall project uses FastAPI as the API layer and PostgreSQL/SQLAlchemy for persistent data management.

5. Working Process

The basic working process of my module is:

Order Creation → Inventory Validation → Picking → Packing → Shipping → Delivery

Whenever inventory is changed, the stock movement is recorded in the database. This provides a reliable history of inventory changes and makes it possible for other modules, particularly the predictive intelligence module, to use warehouse stock-movement information.

6. My Contribution

My contribution focuses on establishing the **core warehouse management functionality**. The module provides the basic operational data required by the remaining modules of the system.

Without accurate order and inventory management, demand forecasting, anomaly detection, robot simulation, and decision support would not have reliable warehouse information to work with.

7. Outcome

The module provides:

- Accurate inventory tracking
- Complete order lifecycle management
- Inter-warehouse transfer support
- Damage and return tracking
- Traceable stock movements
- Reliable warehouse operational data

8. Conclusion

Module 1 provides the foundation of the Smart Warehouse Automation System. By managing orders, inventory, transfers, damages, and returns, it creates a reliable operational layer for the complete platform. My contribution ensures that warehouse activities are properly recorded, validated, and available for use by the project's security, AI, and simulation modules.

REFERENCES

- [1] M. Bodne, V. Patar, S. Sontakke, V. Turankar, and B. V. Kale, “Cloud-Based Inventory Management System: Architecture, Real-Time Analytics, Automation, and Security for Modern Enterprise Supply Chains,” *International Research Journal of Innovations in Engineering and Technology*, vol. 10, no. 5, pp. 241–253, 2026.
- [2] A. Bhargav, A. S. Zope, M. S. Jambhale, N. M. Korde, and H. A. Khan, “IoT Based Industrial Automation,” *International Journal of Engineering Research and Technology*, vol. 5, no. 1, pp. 821–827, 2017.
- [3] R. F. Olanrewaju, “Cloud-Based Warehouse System for Effective Management of Under and Over-stock Hazards,” in *Proc. Int. Conf. on Computer Communication and Engineering (ICCCE)*, 2021, pp. 274–278.
- [4] W. C. Tan and M. S. Sidhu, “Review of RFID and IoT Integration in Supply Chain Management,” *Operations Research Perspectives*, vol. 9, pp. 1342–1348, 2022.
- [5] FastAPI Documentation, “FastAPI: A Modern, Fast Web Framework for Building APIs with Python,” [Online]. Available: <https://fastapi.tiangolo.com>.
- [6] Scikit-learn Developers, “IsolationForest — Scikit-learn Documentation,” [Online]. Available: <https://scikit-learn.org>.
- [7] Google OR-Tools Documentation, “CP-SAT Solver,” [Online]. Available: <https://developers.google.com/optimization>.
- [8] Three.js Documentation, “Three.js — JavaScript 3D Library,” [Online]. Available: <https://threejs.org>.
- [9] Render, “Render Cloud Platform Documentation,” [Online]. Available: <https://render.com/docs>.

APPENDICES

Appendix A – Detailed Calculations

Detailed derivations for the A* cost function, the Manhattan-distance heuristic proof of admissibility on a grid-aligned warehouse layout, and the WAPE/MAE/RMSE/sMAPE forecast-scoring formulas are to be maintained in the project's supplementary calculation notebook and referenced here.

Appendix B – Engineering Drawings / Schematics

The system architecture diagram is provided as Figure 3.1. Additional low-level component and sequence diagrams (API request flow, SSE streaming sequence, notification pipeline) are to be maintained in the project repository's /docs/diagrams directory.

Appendix C – Source Code / Code Repository Information

Only essential code excerpts should normally be included in the report body. The complete source code is maintained in the approved project repository, with the production build deployed at <https://warehouse-cloud-platform.onrender.com>.

Appendix D – Datasheets

Not applicable — no physical hardware components were used in this software-simulated release.

Appendix E – Experimental / Raw Data

Raw pytest output (72 collected, 71 passed, 1 skipped) and the local/production benchmark logs summarised in Table 4.5 are to be retained in the project's /tests and /reports directories for audit purposes.

Appendix F – Ethics / Institutional Approval

Not applicable — the project uses synthetic and simulated datasets only; no human-subject or personally identifiable data was collected.

Appendix G – Project Meeting / Progress Records

To be maintained by the team as dated meeting-minutes records covering the five milestones listed in Section 4.8.

Appendix H – User/Industry Feedback

Not applicable at this stage — the platform has not yet been piloted with an external warehouse operator.

Appendix I – Publications / Patents / Competitions / Product Outcomes

None at the time of this report.

Appendix J – Individual Contribution Evidence

See the Individual Contribution Statement (front matter) and Table 4.8 (Roles & Contribution) for the mandatory team-project contribution mapping.

CAPSTONE PROJECT OUTCOME MAPPING SHEET

(To be completed / verified by the department / faculty assessor.)

Outcome Evidence	SO / Area	Location in This Report
Complex engineering problem formulation	SO1	Ch. 1, Ch. 2
Engineering design under constraints	SO2	Ch. 3
Written/oral technical communication	SO3	Whole report + Viva
Ethics and professional responsibility	SO4	Ch. 3 (3.7)
Teamwork and leadership	SO5	Ch. 4 (4.8)
Experimentation, analysis, engineering judgment	SO6	Ch. 4 (4.3–4.6)
Independent acquisition of new knowledge	SO7	Ch. 5 (5.3)
Standards	SO2	Ch. 3 (3.2)
Sustainability and societal/environmental impact	SO2/SO4	Ch. 3 (3.7)
Risk assessment and mitigation	SO2/SO4	Ch. 3 (3.7)
Security considerations	Relevant SO	Ch. 3 (3.7)
Integrated/system-level engineering	SO1/SO2	Ch. 3 (3.5)
Project planning and management	SO5	Ch. 4 (4.8)
Individual contribution	SO1–SO7	Contribution Statement + Ch. 4 (4.8)